

Solución propuesta Proyecto 1 PLE

Por: Nicolás Hernández Atehortúa

21 de marzo de 2026

Resumen

El presente documento representa una guía para una posible forma de modelar la gramática dada en el enunciado del proyecto 1. Sin embargo, este documento no constituye una solución única y universal para la misma. Por lo cual, puede haber múltiples soluciones igualmente válidas a la presente. Así mismo, este documento a pesar de tener explicaciones en español, hará uso de los nombres en inglés usados en el enunciado para facilitar la comprensión del mismo.

1. ¿Que se busca?

Para este primer proyecto piden definir la gramática de Verilang (El DSL descrito en el documento).

Para esto, se debe de definir los siguientes elementos:

- Elementos no terminales
- Elemento inicial
- Elementos terminales o símbolos (El alfabeto)
- Reglas de producción

Este último debe de ser definido haciendo uso de la notación EBNF.

2. Notación EBNF

La notación EBNF (Extended Backus-Naur Form) va a ser bastante recurrente en este documento, por lo mismo es importante estar familiarizado con la misma. Con este fin, se describe a continuación de qué forma será usada:

- Los paréntesis () son usados para asociar elementos terminales y no terminales. Ej: (AB), (B" c")
- Los símbolos terminales estarán encerrados por comillas dobles y para facilitar su diferenciación se colocaran en color azul. Ej: "def", "while"
- Los elementos no terminales no están encerrados por comillas y para diferenciarlos estarán en color naranja. Ej: Object, Definition

- Las reglas de producción estarán delimitadas por ::= .

Ej: `Object ::= Definition"end"`

- Para representar un o entre varios elementos se usa el |, en medio de los dos elementos a elegir, esto se puede extender o usar con expresiones juntas con paréntesis.

Ej: `Fruta | Verdura`

`("Colombia" Departamento) | "Colombia"`

`Municipio | ("Caldas" Municipio)`

- Para representar cero o más repeticiones de un elemento, se usa un * después de. Ej: `Digito*`, `(Digito Letra)*`
- Para indicar una o más repeticiones de un elemento, se coloca un + al final del mismo. Ej: `Animal+`, `("Alimento: ", Animal)+`
- Para mostrar que un elemento es opcional, se coloca un ? al final. Ej: `Animal Genero?`, `Número "Paridad"?`

3. Gramatica de Verilang

3.1. Elementos listados directamente

En el enunciado del proyecto hay algunos elementos de la gramática que son listados de manera explícita y no requieren de interpretación, los cuales pueden ser tomados como un primer paso para definir la gramática.

Primero que todo, se tienen las palabras reservadas. Estas son definidas como elementos terminales del lenguaje.

Terminales := { "defmodule", "using", "defspace", "defrule", "end", "defoperator", "defexpression", "forall", "exists", "defer" }

Luego están los literales, compuestos por los símbolos usados como operadores y símbolos de puntuación, estos símbolos los añadimos a nuestro conjunto de elementos terminales:

Terminales := { "defmodule", "using", "defspace", "defrule", "end", "defoperator", "defexpression", "forall", "exists", "defer", "*", "/", "-", "+", "**", "%", "<", ">", "<=", ">=", "<>", "=", "and", "or", "neg", "- >", "in", "≡", "=", ".", "(", ")", "[", "]", ":" }

Finalmente, se proporciona la definición de los identificadores o id para abreviar. Siendo que se deben definir como un no terminal que produce una secuencia que siempre empieza por una letra y puede ser sucedida por letras, dígitos o guiones (-). Tal que no son palabras reservadas (Aunque esta restricción no la tendremos en cuenta para este proyecto 1). Así mismo, todas las letras deben ser entendidas como minúsculas. Por lo cual, obtenemos lo siguiente:

`id ::= (a..z)((a..z) | (0..9) | (-))*`

3.2. Módulos

Ahora, al empezar a leer la descripción de Verilang. Es posible notar que los programas de este lenguaje están definidos en módulos donde se dan las especificaciones del programa. Por lo cual, es posible modelar modulo como el elemento inicial. Es decir, q_0 : **Modules**

Luego, se describen los componentes de los programas. Los cuales podemos considerar como elementos no terminales. Entonces

No Terminales := { **Rules**, **Variables**, **Expressions**, **Equations**, **Operators**, **Relations**, **Modules**, **Spaces**, **Atributes**, }

Seguidamente se nos empieza a describir la definición de un modulo. Inician con la palabra "**defmodule**" seguido de su nombre (en general, todos los nombres serán asumidos como identificadores) y terminado con un "**end**" al final de todo el contenido del modulo. Además, es posible importar otros módulos, para esto se usa la palabra clave "**using**" junto al nombre del modulo a importar (El cual se toma como un identificador), terminando con con un salto de linea ($\backslash n$).

Para modelar la posibilidad de importar otros módulos y evitar un ciclo recursivo de un modulo dentro de otro modulo. Se define un elemento no terminal llamado **Import**. El cual puede ser producido por **Modules**. Por otro lado, se define la siguiente regla de producción para **Import**:

Import ::= "**using**" id " $\backslash n$ "

3.3. Espacios

Los espacios son usados para definir los elementos del modulo. Esto entendiendo que en este caso espacio se refiere a un conjunto con algún tipo de estructura (Ej: Espacio vectorial). Ahora bien, su definición empieza con la palabra "**defspace**", continuo al nombre del espacio. Opcionalmente, es posible asignar una relación de sub-espacio. Para esto se añade el operador "<" previo al nombre del espacio más grande del que se toman las propiedades. Esta relación puede ser entendida pero no limitada a la relación que existe entre un espacio vectorial y un sub-espacio vectorial. Finalmente, se debe colocar "**end**" para terminar la definición del espacio.

Para modelar esto, se usa el elemento no terminal definido anteriormente **Spaces**, el cual puede ser producido por **Modules**. Mientras tanto, **Spaces** posee la siguiente regla de producción:

Spaces ::= "**defspace**" id ("<" id)? "**end**"

3.4. Operadores

Para definir un operador, se empieza con la palabra "**defoperator**". Sucesivo a ello se coloca el nombre del operador y dos puntos ":". Seguidamente, se coloca el dominio del operador, una flecha ">" y el rango del mismo. Se entiende por dominio, el conjunto de los elementos que recibe el operador. Mientras que el rango, hace referencia al conjunto de los elementos devueltos por el operador. Por lo cual, se definen un no terminales **Domain**. No se define uno diferente para rango, puesto que ambos hacen referencia a un conjunto de elementos. Para cerrar se coloca la palabra "**end**".

Cabe recalcar que, Verilang usa una notación Curryng para la definición de operadores. Esto quiere decir, que los operadores son funciones de un solo parámetro y de un solo retorno. Para la representación de una función de varias funciones se hace uso de composición de funciones. Para ejemplificarlo se usará python.

Suma de tres números

Definición sin Curryng:

- Definición de la función

```
def suma_Tres_Numeros(a,b,c):
    return a + b +c
```

- Llamado de la función

```
resultado = suma_Tres_Numeros(a,b,c)
```

Definición con Curryng:

- Definición de la función

```
def suma(a):
    def sumar_b(b):
        def sumar_c(c):
            return a + b +c
        return sumar_c
    return sumar_b
```

- Llamado de la función

```
intermedio = suma(1) #Devuelve una funcion
primer_resultado = intermedio(5) #Devuelve una funcion
resultado_final = primer_resultado(8) #Devuelve la suma de 1+5+8
print(resultado_final) #14
```

Para ajustar este comportamiento, se define la siguiente regla de producción.

Operator ::= "defoperator" id ":" Domain "– >" (Domain | (Domain "– >" Domain))* "end"

Note que en el caso de (Domain "– >" Domain)*, se hace referencia al dominio y rango de otra función. Así mismo, cabe recalcar que el rango de la función inmediatamente anterior corresponde al dominio de la siguiente.

3.5. Expresiones

Para definir una expresión, se empieza con la palabra "defexpression". Luego se procede a describir una expresión lógica. Al inicio es posible notar su similitud con los condicionales en programación. Aunque en este contexto, son utilizados para indicar que alguno de los elementos definidos en el modulo cumple una propiedad o que queremos la parte de dichos elementos que cumplan con la o las mismas. Con elementos se hace referencia a Espacios, Operadores, Relaciones.

Antes de continuar, se debe realizar una aclaración sobre los operadores de primer orden. Si bien es cierto que es posible usar cualquier operador de primer orden. Para efectos prácticos, se explicaran solo aquellos con los cuales es posible establecer una equivalencia con los elementos terminales del lenguaje.

$$\forall \rightarrow \text{"forall"} \quad (\text{Para todo}) \quad (1)$$

$$\exists \rightarrow \text{"exists"} \quad (\text{Existencia}) \quad (2)$$

$$\neg \rightarrow \text{"neg"} \quad (\text{negación}) \quad (3)$$

$$\wedge \rightarrow \text{"and"} \quad (\text{conjunción}) \quad (4)$$

$$\vee \rightarrow \text{"or"} \quad (\text{disyunción}) \quad (5)$$

$$\rightarrow \rightarrow \text{"->"} \quad (\text{implicación}) \quad (6)$$

$$\leftrightarrow \rightarrow \text{"<>"} \quad (\text{bicondicional}) \quad (7)$$

$$\in \rightarrow \text{"in"} \quad (\text{Pertenencia}) \quad (8)$$

Habiendo aclarado lo anterior. Se continua con la definición de la expresión logica. Primeramente, se puede optar por cuantificar las variables ($\forall, \exists$). Después se escribe la definición de la expresión, lo cual consiste en la siguiente estructura:

$$\text{Objeto}_1 \text{ operador } \text{Objeto}_2$$

Donde los objetos 1 y 2 pueden ser un Espacios, Operadores, Relaciones. Adicionalmente, en el caso del objeto 1, es posible declara una variable para indicar el elemento de uno de los objetos anteriormente mencionados. Una vez declarado dicha variable, también es posible usarla como objeto 2. Los operadores pueden ser cualquiera de los listados anteriormente, ademas de los siguientes:

$$x = y \rightarrow \text{"="} \quad (\text{igualdad}) \quad (9)$$

$$x \neq y \rightarrow \text{"neg(x=y)"} \quad (\text{desigualdad}) \quad (10)$$

$$\geq \rightarrow \text{">="} \quad (\text{Mayor o igual que}) \quad (11)$$

$$\leq \rightarrow \text{"<="} \quad (\text{Menor o igual que}) \quad (12)$$

$$\equiv \rightarrow \text{"\equiv"} \quad (\text{Congruencia/Equivalencia}) \quad (13)$$

Por otro lado, después es posible incluir atributos (Los cuales serán revisados más adelante). Finalmente, para completar la definición de la expresión. Se coloca la palabra **"end"**. Para modelar todo esto, se define que **Modules** puede generar a **Expressions**. Luego, la regla de producción de **Expressions** es la siguiente:

$$\text{Expressions} ::= \text{"defexpression"} \text{ LogicalExpressions } \text{"end"}$$

$$\text{LogicalExpressions} ::= \text{"(" ("forall" | "exists")? "neg"? id ("and" | "or" | ">" | "<>" | "in" | "in" | "=" | "neg(x=y)" | ">=" | "<=" | "\equiv") (id | LogicalExpressions) ("." LogicalExpressions)? Attributes ? ")"}$$

Nota: los paréntesis se añaden por el ejemplo dado en el enunciado, aunque estos no se mencionen explícitamente. Lo mismo aplica para el punto. Que se usa para indicar que se continuara con otra expresión lógica. Adicionalmente, note que el operador de negación usa solo un argumento, por lo cual engloba toda la expresión en caso de uso, siguiendo la estructura neg objeto1 operador objeto 2, sin paréntesis.

3.6. Reglas

Las reglas son usadas para definir una regla entre dos operadores. Lo cual se interpreta como una equivalencia. Para la definición de una regla, se empieza con la palabra "defrule". Seguido de la aplicación de un operador y sus parámetros, todo entre paréntesis. Se usa "–>" para conectarlo con la aplicación de otro operador con sus parámetros, también entre paréntesis. Para terminar, se coloca la palabra "end".

Para modelar las reglas, se define que **Modules** puede generar **Rules**. Luego, se define la siguiente regla de producción:

Rules ::= "defrule" "(" id (id)* ")" "–>" "(" id (id)* ")" "end"

Nota: El primer id hace referencia al nombre del operador, los siguientes hacen referencia a los parámetros

3.7. Atributos

Los atributos son una lista de nombres operadores entre corchetes, separados por un espacio. Estos nombres pueden estar seguidos por dos puntos y su dominio o valor. En la lista también pueden haber operadores de un espacio mayor o propiedades.

Para describir esto, se define que **Modules** puede producir **Attributes**. Luego, la regla de producción para **Attributes** es:

Attributes ::= "[" (id (":" Domain | Id)?)* "]"

3.8. Variables

Las variables pueden ser declaradas dentro de una expresión. Pero también pueden ser definidas por aparte empezando por la palabra "defvar" seguida del nombre, dos puntos y el dominio de la misma en ese mismo orden. Es posible declarar varias variables de una vez, separando cada variable con un espacio. Para finalizar, se coloca la palabra "end"

Para modelar esto, se define que **Modules**, puede producir "**Variables**". No se define en este apartado una variable en una expresión, pues ya se definió en dicha sección. Se define la siguiente regla de producción para **Variables**:

Variables ::= "defvar" id ":" Domain ((", " id ":" Domain)*)? "end"

Nota: La coma se añade como un separador adicional para concordar con el ejemplo dado en el enunciado.

3.9. Otros

Una vez ya visto los elementos principales de la gramática, hay algunos elementos que todavía faltan por definir o que no se explicaron a cabalidad previamente. Primero que todo, se define la regla de producción para **Modules**, el cual jugará el rol de ser la primera regla que se debe aplicar.

Modules ::= "defmodule" id (Imports | Rules | Variables | Expressions | Equations | Operators | Relations | Spaces | Attributes)* "end"

Luego, el no terminal **Domain**. El cual se usa para referirse a un conjunto de valores. En general, los dominios pueden ser módulos declarados. Puesto que cada modulo describe un conjunto con propiedades. Así mismo, también es posible asumir que hace referencia a los tipos de datos tradicionales. Sin embargo, se limitaran a aquellos que van más acorde con el proposito del lenguaje. Por lo cual, se define la siguiente regla de producción para **Domain**

Domain ::= **id** | **bool** | **int** | **real**

Nota: El dato real hace referencia a cualquier número real.

4. Listado de elementos

4.1. Elemento inicial

q_0 : **Modules**

4.2. Elementos no terminales

No Terminales := { **Rules**, **Variables**, **Expressions**, **Equations**, **Operators**, **Relations**, **Modules**, **Spaces**, **Atributes**, **Imports**, **id**, **Modules**, **Domain**, **LogicalExpressions**, **bool**, **int**, **real** }

4.3. Elementos terminales

Terminales := { **"defmodule"**, **üsing**, **"defspace"**, **"defrule"**, **"end"**, **"defoperator"**, **"defexpression"**, **"forall"**, **"exists"**, **"defer"**, **"*"**, **"/"**, **"-"**, **"+"**, **"**"**, **"%"**, **"<"**, **">"**, **"<="**, **">="**, **"<>"**, **"="**, **"and"**, **"or"**, **"neg"**, **"- >"**, **"in"**, **"≡"**, **"="**, **"."**, **"("**, **)"**, **"["**, **"]"**, **":"** }

4.4. Reglas de producción

Modules ::= **"defmodule"** **id** (**Imports** | **Rules** | **Variables** | **Expressions** | **Equations** | **Operators** | **Relations** | **Spaces** | **Atributes**)^{*} **"end"**

Import ::= **"using"** **id** **"\n"**

Spaces ::= **"defspace"** **id** (**"<"** **id**)[?] **"end"**

Operator ::= **"defoperator"** **id** **":"** **Domain** **"- >"** (**Domain** | (**Domain** **"- >"** **Domain**)^{*})^{*} **"end"**

Expressions ::= **"defexpression"** **LogicalExpressions** **"end"**

LogicalExpressions ::= **"("** (**"forall"** | **"exists"**)[?] **"neg"**[?] **id** (**"and"** | **"or"** | **"- >"** | **"<>"** | **"in"** | **"in"** | **"="** | **"neg(x=y)"** | **">="** | **"<="** | **"≡"**) (**id** | **LogicalExpressions**) (**"."** **LogicalExpressions**) | **Atributes**)[?] **"")**

Rules ::= **"defrule"** **"("** **id** (**id**)^{*} **"")** **"- >"** **"("** **id** (**id**)^{*} **"")** **"end"**

Atributes ::= **"["** (**id** (**":"** **Domain** | **Id**)[?])^{*} **"]"**

Variables ::= **"defvar"** **id** **":"** **Domain** (**"("** **id** **":"** **Domain**)^{*})[?] **"end"**

Domain ::= **id** | **bool** | **int** | **real**

`id ::= (a..z)((a..z) | (0..9) | (-))*`

`bool ::= "true" | "false"`

`int ::= (0..9)+`

`real ::= (0..9)+ "," (0..9)*`

Nota: El elemento no terminal equations no tiene regla de producción puesto que en el enunciado no se especificó cuál era su comportamiento.

5. Ejemplos

5.1. Espacios

```
defspace "VectorSpace"
defspace "PolynomialSpace" < "VectorSpace"
defspace "MatrixSpace"
defspace "RealNumbers"
defspace "ComplexPlane"
defspace "GraphSpace"
defspace "FiniteField" < "RealNumbers"
defspace "ProbabilitySpace"
```

5.2. Operadores

```
defoperator "max" : int -> int -> int end
defoperator "min" : int -> int -> int end
defoperator "abs" : int -> int end
defoperator "sqrt" : real -> real end
defoperator "add" : real -> real -> real end
defoperator "mul" : real -> real -> real end
defoperator "pow" : real -> int -> real end
```

5.3. Expresiones

```
defexpression (forall x in Set . (x "max" 0)) end
defexpression (exists y in Set. (y "min" 10)) end
defexpression ("abs" <= 0) end
defexpression ("sqrt" <= 0.0) end
defexpression (p and q) end
defexpression (p or q) end
defexpression (not r) end
defexpression (forall x in "Element" . (forall S in "Set" . (forall U in "
  ↪ Set" . x "isIn" (S "union" U) = (x "isIn" S or x "isIn" U)))) end
defexpression ( forall a in "Element" . (forall B in "Set" . (forall C in
  ↪ "Set" . a "isIn" (B "intersection" C) = (a "isIn" B and a "isIn" C)
  ↪ ))) end
```


5.4. Reglas

```
defrule ("add" x y) -> ("sum" x y) end
defrule ("mul" x y) -> ("product" x y) end
defrule ("neg" x) -> ("minus" x) end
defrule ("and" a b) -> ("conj" a b) end
defrule ("or" a b) -> ("disj" a b) end
defrule ("eq" a b) -> ("equals" a b) end
defrule ("lt" a b) -> ("lessThan" a b) end
defrule ("gt" a b) -> ("greaterThan" a b) end
defrule ("concat" s1 s2) -> ("join" s1 s2) end
defrule ("pow" b e) -> ("exponentiate" b e) end
```

5.5. Atributos

```
[ "commutative":bool ]
[ "associative":bool ]
[ "identity":int ]
[ "inverse":int ]
[ "arity":int ]
[ "type":1, "parity":2]
[ "type":2 ]
[ "type":false ]
```

5.6. Variables

```
defvar "x":int end
defvar "y":int end
defvar "z":int end
defvar "flag":bool end
defvar "msg":string end
defvar "matrix":MatrixSpace end
defvar "poly":PolynomialSpace end
defvar "vec":VectorSpace end
defvar "prob":ProbabilitySpace end
defvar "counter":int end
```

6. Módulo completo

```
defmodule "DemoModule"
  using "BaseTypes"
  using "MathOps"

  defspace "VectorSpace"
  defspace "MatrixSpace" < "VectorSpace"

  defoperator "add" int -> int -> int end
```

```

defoperator "mul" int -> int -> int end
defoperator "neg" int -> int end
defoperator "eq" int -> int -> bool end

defvar "a" : int, "b" : int, "v" : VectorSpace end

defexpression (forall a neg (a "lt" "0")) end
defexpression (exists b (b "eq" "5")) end

defrule (add a b) -> (sum a b) end
defrule (neg a) -> (minus a) end

[ "commutative" : bool ]
[ "type" : "int" ]
end

```

Nota: Las palabras entre comillas hacen referencia a identificadores.